KRNL v0.3

ESP32 Wi-Fi Recon HUD

Touchscreen reconnaissance, management-frame monitoring, IR diagnostics and local SD records.

KRNL is a standalone Arduino-style firmware for a classic ESP32 with an ILI9341 display, XPT2046 resistive touch controller, microSD storage and a TSOP1838B IR receiver. It discovers access points, keeps a bounded history, displays signal and channel information, and records observations locally.

> **Source status:** This README documents the newer, complete-looking v0.3 attachment. It contains setup() and loop() and draws its boot skull procedurally; skull_boot.h is no longer required. Features are verified by source inspection, not by a successful compilation or physical hardware test.

What is implemented

Six navigation screens; asynchronous Wi-Fi discovery; a 40-entry AP cache; RSSI history and a watchlist; heuristic guard alerts; SD logs and CSV exports; selective management-frame PCAP output; four-point touch calibration; manual time entry; a local four-digit access PIN; IR pulse-burst diagnostics; and a small animated KRNL-17 avatar.

Two distinctions that matter

Scanning is not configured as passive. startScan() calls WiFi.scanNetworks(true, showHidden). The first argument enables asynchronous operation; the omitted passive argument defaults to false in the documented API [W1]. The receive-only packet-monitor path is separate. The repeated "passive" UI/log labels overstate the scan configuration.

BLE is optional and disabled. KRNL_ENABLE_BLE is 0. A BLE scanning implementation exists behind the compile-time switch; it is not part of the default build. This is different from an unimplemented future feature.

Inside this README	Page
Components and exact wiring	2
Architecture and feature behavior	3
Build, upload and first boot	4
Everyday use and controls	5
SD files, timestamps and capture format	6
Limitations, known issues and proposed work	7
Troubleshooting and source references	8

Prepared 4 November 2026. Source: Pasted text.txt, firmware banner "KRNL firmware v0.3". The earlier 683-line fragment is superseded for this document.

Components & wiring

Use the v0.3 constants below. GPIO numbers are ESP32 signals, not physical header positions. The earlier attachment used different TFT and touch pins. [S1]

Component	Role / requirement
Classic ESP32 development board	Firmware target; exposes the GPIOs below. Exact board and flash capacity are not specified.
ILI9341 SPI TFT	320 x 240 landscape UI, rotation 1; display starts at 10 MHz SPI.
XPT2046 controller / resistive panel	Touch rotation 1; dedicated CS and IRQ. May be integrated with the TFT.
SPI microSD interface and card	Local persistence; initialization tries 4 MHz, then 1 MHz. Core UI can run when mounting fails.
TSOP1838B IR receiver	Demodulated active-low receiver; OUT is sampled by a CHANGE interrupt.
USB power/data cable and interconnects	Needed for programming and assembly; power wiring is module-specific.

ESP32 GPIO	Connect to	Source constant
18	TFT SCK, touch T_CLK, SD SCK	PIN_SPI_SCK
19	TFT SDO (if used), touch T_DO, SD MISO	PIN_SPI_MISO
23	TFT SDI, touch T_DIN, SD MOSI	PIN_SPI_MOSI
27	TFT CS	PIN_TFT_CS
26	TFT D/C	PIN_TFT_DC
25	TFT RESET	PIN_TFT_RST
33	Touch T_CS	PIN_TOUCH_CS
32	Touch T_IRQ	PIN_TOUCH_IRQ
13	SD CS	PIN_SD_CS
21	IR receiver OUT	PIN_IR_RECEIVER

Electrical details not supplied by code: VCC, ground-pin positions, backlight wiring, regulator requirements and the receiver package pin order. Connect common grounds and use each actual module's documentation for supply voltage and ESP32-compatible signal levels. Do not infer a package pin order from the receiver name.

Shared bus: SCK/MOSI/MISO are common; each SPI peripheral has its own chip select. setup() drives all three CS lines high before SPI.begin(). releaseSPIForSD() deselects the display and touch before storage operations. There is no backlight-control GPIO in this firmware. [S1, S2, S10]

Architecture & behavior

Layer	Implemented flow
Startup	Serial and packet queue → SPI/display/touch/IR → saved settings → boot graphic → calibration if needed → time editor → SD mount → PIN setup/unlock → Wi-Fi → first screen.
Main loop	Services scanner, packet monitor, IR, touch and SD logs; then redraws dirty content or animates radar/log waveform/avatar. Header refresh is nominally 1 s; loop ends with delay(5).
Radio input	Scan results populate the AP cache. Monitor callback filters selected management subtypes into a 12-item FreeRTOS queue; the main loop processes it and writes eligible frames.
State and outputs	RAM holds APs/history/logs; Preferences namespace "krnl" holds settings, calibration, PIN hash, watchlist and counters; SD holds session records, baseline and exports.

Recon and guard [S3, S4]

AP identity is BSSID. At most 40 entries are retained; a new AP replaces the oldest last-seen slot when full. Each entry holds SSID, BSSID, RSSI, channel, authentication, sightings, change counters and a 16-sample RSSI ring. Visible entries are sorted by RSSI; missing APs remain eligible for display for about 120 seconds when the list is rebuilt. Recon shows five rows per page.

Automatic scanning defaults on with a 10-second base interval. Settings cycle 5 / 10 / 20 / 30 seconds. Adaptive mode schedules 5 seconds after new/channel/security events and at least 30 seconds after five stable scans. These intervals run from scan completion, so they are not a fixed scan-to-scan period.

Guard can alert for post-baseline newcomers, watched AP disappearance and anomaly scores. The baseline is saved after three completed scans, using active APs from that save point; it is not a union of all three scans. Baseline persistence needs SD. The watchlist holds up to eight BSSIDs in Preferences.

Score interpretation

The 0-100 heuristic adds 35 for open authentication, 30 for same-SSID/different-security matches, up to 20 for security changes, up to 10 for repeated channel changes, 10 for at least three other same-SSID cache entries, and 5 for a missing AP. Warnings begin at 35. These flags are observations, not proof of an intrusion.

Monitor, IR and display [S2, S7, S8]

Packet mode pauses scanning and receives probe requests/responses, beacons, disassociation and deauthentication metadata. Hopping defaults on, advancing across channels 1-13 every 700 ms. Only beacons and probe responses enter PCAP. IR records edge counts and min/max high/low durations after more than 50 ms of silence, rejecting bursts below four edges.

Radar uses BSSID-hashed angles and RSSI-derived radius, with at most 24 plotted APs. It does not measure bearing or physical distance. The log waveform is decorative; KRNL-17 reflects software mood states.

Build & first boot

1. Prepare the sketch

Save the supplied source as KRNL_v03.ino inside a matching KRNL_v03 sketch folder. This attachment includes its own boot graphics and needs no external skull bitmap header. Keep the original source unchanged as a reference.

2. Install the build dependencies

Dependency	Why it is needed
Espressif Arduino-ESP32 board package	Arduino, SPI, WiFi, Preferences, SD, ESP system/Wi-Fi APIs and FreeRTOS headers. Choose the actual classic ESP32 board.
Adafruit GFX Library	Graphics primitives; allow its declared dependencies to install.
Adafruit ILI9341	Display driver used by the hardware-SPI constructor.
XPT2046_Touchscreen	Touch driver and TS_Point type.
BLE support (optional)	Uses the ESP32 BLE API when KRNL_ENABLE_BLE=1; its result-pointer form matches the 3.x API change [W2].

No exact board-package or library versions, lockfile, compiler report or binary is supplied. Record versions after a successful build. Arduino-ESP32 3.x is the appropriate starting point for the optional BLE code, but this document does not certify a specific release.

3. Select board, storage profile and port

Keep KRNL_ENABLE_BLE=0 for the first build. Select the connected board and serial port, compile, and check the reported application size against the chosen partition. The source recommends "Huge APP (3MB No OTA/1MB SPIFFS)" when enabling BLE; use it only if supported by the board's flash layout. The source's approximate BLE size comment is not a measured build result.

4. Wire, upload and inspect startup

Wire against page 2 and the component datasheets. Upload the sketch and open Serial Monitor at **115200 baud**. On first boot, press and hold each of four calibration targets, then release. Short or invalid samples repeat. Calibration version 3 is saved in Preferences.

5. Complete the startup prompts

The manual clock editor appears on every boot. Use H-/H+ and M-/M+, then SAVE or CANCEL. Time advances only while powered; canceling can leave an unset or stale saved clock. The SD card mounts next, followed by creation and confirmation of a four-digit PIN, or entry of the existing PIN. Five failed boot unlocks trigger a 10-second temporary lockout. [S5, S6, S10]

Initial acceptance check: Confirm readable display and accurate touch, inspect Serial/LOGS for SD status, wait for a scan result, open one AP, and check that a session log and scan CSV appear on the card. These are recommended checks to perform on the device; they have not been executed for this README.

Using the HUD

Screen / control	What to do and expect
HOME	Shows cache, open-network and completed-scan counts plus anomaly flags. OPEN RECON MODULE opens the results list. "ONLINE" is a UI state, not confirmation of an Internet connection.
RECON	Tap > to cycle pages. Tap an AP row to open DETAILS. The list is sorted by RSSI and may include recently missing entries. W marks a watched BSSID.
DETAIL / LIVE	INFO displays metadata, status and changes. LIVE displays the 16-sample RSSI history. ADD WATCH / WATCHING toggles watchlist membership. SIM INCIDENT cycles three labeled synthetic incidents and writes a simulated alert if SD is ready.
LOGS	Shows up to eight newest entries from a 30-entry RAM ring, plus SD status and queue depth. CLEAR resets the RAM log view; it does not delete existing SD records.
TOOLS	The top-right button cycles RADAR → CHANNEL → DIAG → IR → PACKET. Radar blips open AP details. RESCAN starts a manual scan in radar/channel modes.
SET	The top-right page selector cycles scanning/time, system/hardware and live/guard settings. Most toggles persist immediately in Preferences.

Packet monitor workflow [S2, S9]

Open TOOLS and cycle to PACKET. Tap START and enter the access PIN. CH- and CH+ choose a fixed channel; HOP/FIXED toggles automatic hopping. Counters show accepted management traffic, with deauth/disassoc observations and the last frame metadata. Tap STOP when finished. Changing tools mode or leaving TOOLS stops monitoring, and automatic scanning can resume afterward.

SD is optional for the live counters but required for capture files. The on-screen "SD READY" label reflects sdReady; it does not prove every PCAP write succeeded. Monitoring does not capture data/control payloads. Do not interpret absent events as proof that none occurred on other channels.

IR and optional BLE

In IR mode, point a compatible remote at the receiver and press a button. The view shows pulse-burst statistics; CLEAR resets displayed counters. It does not decode button names, protocols or command bytes and does not transmit IR. In DIAG, BLE SCAN reports a disabled-build message by default; when compiled in, it performs a four-second passive BLE scan, stores up to ten results and displays the first device summary.

Settings and deletion

SET page 1 controls automatic scan, hidden-result inclusion, base interval and manual time. Page 2 offers radio power saving, touch calibration, AP cache clearing and WIPE KRNL DATA. Page 3 controls adaptive scan and guard, and resets a ready baseline. The wipe requires PIN plus confirmation, deletes known KRNL SD records, clears runtime caches and starts a new session; Preferences settings, PIN and watchlist remain. [S6, S9]

SD records & exports

All v0.3 records live under **/krnl**. Session and capture IDs are 16-bit counters saved in Preferences. The four-digit filenames are minimum-width formats, not a four-digit maximum. The earlier /logs/session_*.txt format does not apply. [S2, S3]

Path under /krnl	Contents / update behavior
logs/S####.log	Session text events: clock, uptime milliseconds, category, message.
logs/S####.meta	Session ID, reset reason, CPU frequency, flash size and card capacity.
scans/S####.csv	Per completed scan: clock, uptime, scan number, SSID, BSSID, RSSI, channel, security, present/missing status, sightings, channel/security changes and anomaly score.
latest.csv	Replaced after each scan; active visible APs with SSID, BSSID, RSSI, channel, security and anomaly.
summaries.csv	Scan result/visible/open/warning counts, duration and free heap.
alerts.csv	Changed anomaly scores at or above 35 when guard is on, plus explicitly labeled simulated incidents.
security.csv	PIN and wipe-related SECURITY events with clock, uptime and session.
ir.csv	Burst number, edges and min/max low/high durations in microseconds.
monitor_events.csv	Rate-limited deauthentication/disassociation events: channel, RSSI, transmitter, BSSID and reason code.
captures/C####.pcap	Beacon/probe-response frame prefixes only; 128-byte snap length, raw 802.11 link type 105.
baseline.csv	BSSID, SSID, security, channel snapshot. Reload currently uses BSSID membership only.
config.txt	Readable settings snapshot and flags; PIN value is not exported. This is not an implemented settings-import format.

Timing, retention and interpretation

Text events queue in 16 fixed slots of 128 bytes; overflow drops the oldest queued line. Routine flushing is every two seconds, with forced flushes at selected events such as scan completion. Messages are first limited to 64 characters in the RAM log entry. Abrupt power loss can lose queued data, and write return values are not consistently checked.

Clock strings are manual time-of-day with no date or timezone. PCAP timestamps are micros() uptime, not the manual clock or Unix time; the 32-bit microsecond counter wraps after about 71.6 minutes. Captures are truncated, channel hopping samples only one channel at a time, and the packet queue can overflow. STOP resets any remaining queued frames.

SSIDs and BSSIDs are stored in clear text. There is no SSID masking in this version. Copy records after a completed flush and orderly shutdown; no dedicated safe-eject control is implemented. The PIN is a local interface gate, not SD encryption.

Limitations & known issues

These observations are from static review of the supplied source. The firmware was not compiled, flashed or tested on hardware during document preparation.

Finding	Effect / recommended follow-up
Scan mode label mismatch [S4, W1]	WiFi.scanNetworks(true, showHidden) uses the default active-scan setting. If passive discovery is intended, explicitly pass the third argument true and verify behavior; otherwise correct the labels.
Repeated missing-watch alerts [S4]	missedScans increments only when an entry was active. After the first miss it can remain 1, so the ==1 alert condition fires on subsequent scans. Track present-to-missing transitions or increment consecutive misses consistently.
No application scan watchdog [S4]	serviceScanner() handles completion/failure but does not enforce an elapsed-time timeout. Add bounded recovery for a scan that remains running.
Bounded cache and heuristic alerts [S3, S4]	Only 40 APs are retained. Stale same-SSID entries affect comparisons; legitimate open networks score 35. Baseline tracks membership, not ongoing comparison of saved security/channel values.
UI overflow in details/logs [S7]	The 11th details field starts at y=201 with navigation at y=212; content is tight. Eighth log row at y=187 intersects the waveform region beginning at y=190. Inspect on hardware and reserve separate regions.
Storage and capture reliability [S2, S6]	Many writes/removals lack result checks. Wipe success is not verified file by file. Session/capture counters can wrap and reuse paths. sdWriteError can be cleared by a later successful operation.
Blocking interactions [S5, S6, S8]	Calibration, time/PIN dialogs and the optional four-second BLE scan block normal loop servicing. They are not background tasks.
PIN and timestamps [S6, S2]	Four-digit PIN uses a device-salted 32-bit non-cryptographic hash; no file encryption. Manual clock has no battery-backed continuity or date, and PCAP uptime wraps.

Optional, absent and proposed are different

Optional implemented: BLE scanning behind KRNL_ENABLE_BLE. **Absent:** GPS/location, real direction finding, IR protocol decoding/transmission, an RTC/NTP time source, cloud sync, OTA update flow and full traffic capture. The source does not commit to a roadmap for these features.

Proposed maintenance, not implemented: fix the scan-label mismatch and missing-watch repetition; add a scan watchdog; validate SD operations and capture state; make timestamps explicit; verify layouts and pin electrical compatibility; pin dependency versions after a successful build. This README does not modify the firmware.

Troubleshooting & references

Symptom	Checks grounded in this version
Blank display / touch unavailable	Confirm the new GPIO map, common ground and module power/backlight requirements. TFT CS/DC/RST are 27/26/25; touch CS/IRQ are 33/32. Both drivers use rotation 1.
Calibration repeats / taps misalign	Press and hold each target, then release. Samples require pressure above 80, at least four samples and raw-axis spans above 500. Recalibrate from SET page 2.
No scans or results	Finish startup prompts, enable automatic scan or use RESCAN, and stop packet monitoring. Inspect Serial at 115200 and scan failure count in DIAG. The target is 2.4 GHz.
SD offline / missing exports	Inspect card/module wiring and GPIO13 CS. Startup retries at 1 MHz after 4 MHz. Verify the card is readable by the installed SD stack and has free space; no hot-plug remount service is present.
BLE BUILD DISABLED / build too large	Default behavior is deliberate. Enable the flag only with a suitable application partition. For BLE return-type errors, check Arduino-ESP32 API compatibility [W2].
IR stays LOW / no burst	Check actual receiver OUT/GND/VS pin order and supply compatibility. GPIO21 expects active-low demodulated output; isolated edges are discarded.
PCAP appears sparse or has odd dates	Only beacon/probe-response prefixes are saved. Check dropped-frame count, channel selection and SD writes. Timestamps are uptime, not calendar time.
Clock or alert seems incorrect	Clock resets to a saved starting value; set it at boot. Treat anomaly scores as heuristics and distinguish SIM incidents from observed events.

Code evidence map

Line numbers refer to the newer **Pasted text.txt** attachment, counted from #include <Arduino.h> as line 1. Function names remain useful if the file is reformatted.

S1 Lines 1-86: includes, target, pins and limits. **S2** 336-856: IR ISR, monitor callback, SD initialization, queued logs and packet capture. **S3** 857-1346: exports, cache helpers, anomaly scoring, baseline/watchlist and simulation. **S4** 1350-1498: startScan(), finishScan(), serviceScanner(). **S5** 1502-1789: settings, touch calibration and clock. **S6** 1791-2072: PIN, confirmation and data wipe. **S7** 2087-2562: avatar, navigation, details/logs, channels and radar. **S8** 2564-2820: BLE, diagnostics, IR/packet tools and radar animation. **S9** 2834-3153: settings and touch actions. **S10** 3158-end: procedural boot graphic, setup() and loop().

W1 Espressif Arduino-ESP32 Wi-Fi API: scanNetworks() argument defaults.
W2 Espressif migration guide, 2.x to 3.0: BLEScan result-pointer API change. Accessed 4 November 2026.

Source SHA-256: 0d075a0d8202186b50d11674496f52e571ea4cb621bff4f60a4ea646f92b7b9b